

Lv6 · Agents & Skills

여러 단계 작업을 자율 처리하는 AI 에이전트 설계

총 소요: 75분 (강의 60 + Q&A 15)

구성: 12 슬라이드 · 5 원칙 + 4 실전 패턴

대상: 시니어 개발자·아키텍트

출처: Anthropic *Building Effective Agents* (2024.12) 백서 풀 커버 + Skills 공식 문서

강조점: 사령관님 정책 — 원칙·철학 디테일 + 풍부한 사례

S01 · 인트로

3분

01/12

강사 멘트

"Lv5까지는 한 번의 응답을 만드는 AI였습니다. 오늘은 여러 단계를 자율적으로 끝까지 끌고 가는 AI를 설계합니다. 단, Anthropic 백서가 강조하는 핵심: **가능하면 Agent를 쓰지 마라**. Workflow가 답일 때가 90%."

→ 로드맵

4 블록

Block A 정의·철학 / B 5 패턴 / C Skills + 4 시나리오 / D 안전·모범

→ 원칙 1

Anthropic의 핵심 정의

"Workflow는 사람이 코드로 흐름을 고정. Agent는 LLM이 흐름을 결정." — Building Effective Agents (Anthropic, 2024.12)

풍부한 사례

CASE 1 · Workflow 적합 작업

"고객 이메일 받으면 → 분류 → 답장 초안 → 사람 확인" — 매번 같은 순서. 코드로 명시 가능. **Workflow 선택.**

CASE 2 · Agent 적합 작업

"이 고객 문제 해결해줘" — 어떤 도구를 언제 쓸지 case by case. LLM이 결정 필요. **Agent 선택.**

CASE 3 · 잘못 선택한 사례

한 스타트업이 "PDF 요약"을 Agent로 구현 → 무한 루프로 1일 만에 API 비용 \$500 발생. **단순 작업은 단순 호출이 정답.**

CASE 4 · Workflow 디버그 이점

단계별 게이트 검증 → 한 단계 실패 시 즉시 중단. 사고 추적 분 단위.

CASE 5 · Agent 위험성

예측 불가능한 흐름 → 사고 시 원인 추적 어려움. Trace 인프라 없으면 절대 도입 X.

Q. "Agent가 멋있어 보이는데?"

A. "멋"이 도입 이유면 무조건 실패. **해결할 문제가 진짜 예측 불가능한가가 도입 기준.**

→ 5 패턴

Anthropic 백서의 5가지

1. **Prompt Chaining** — 단계별 변환
2. **Routing** — 분류 후 분기
3. **Parallelization** — 동시 처리
4. **Orchestrator-Workers** — 마스터가 분해·위임
5. **Evaluator-Optimizer** — 생성+평가 반복

풍부한 사례

CASE 1 · Chaining

이메일 → 분류 → 답장. 한 단계 결과를 다음 단계 입력으로.

CASE 2 · Routing

"긴급/일반/스팸" 분류 → 적절한 처리기로. 비용 효율 (간단한 건 Haiku).

CASE 3 · Parallel

긴 보고서 3개 동시 요약 → 합치기. 속도 3배.

CASE 4 · Orchestrator

"여행 계획" → 마스터가 [숙소·항공·일정] 분배 → 워커 LLM이 각각 → 통합.

CASE 5 · Evaluator-Optimizer

번역 → 평가자가 점수·피드백 → 재번역 → 합격까지 반복. 품질 ↑ 비용 ↑.

실전 예시

CASE 1 · 이메일 자동 응대 Chain

```
Step 1: classify(email) → {category, urgency}
Step 2: search_knowledge(category) → [relevant_docs]
Step 3: draft_reply(email, docs) → reply_text
Step 4: human_review (사람 확인) → send
```

CASE 2 · 게이트 검증

```
if classify_result.category == "spam":
    archive_and_stop() # 다음 단계 건너뛰
if classify_result.urgency == "high":
    notify_human() # 사람 즉시 알림
```

CASE 3 · Routing 비용 최적화

일반 문의 → Haiku(\$0.25/Mtok), 복잡한 문의 → Sonnet(\$3/Mtok). 월 비용 60% ↓.

CASE 4 · 한국 사례 - 인쇄소 응대

인천 H인쇄소: 일일 50건 거래처 메일. Routing으로 "재발급/견적/배송/기타" 분류 → 각각 다른 템플릿. 처리 시간 1/5.

CASE 5 · 실패 처리

각 단계마다 try/except. 실패 시 사람 알림 + 컨텍스트 보존.

→ Parallel·Orchestrator

풍부한 사례

CASE 1 · Parallel – 다중 관점

긴 계약서 → 3개 LLM 동시: [법무 위험·재무 영향·운영 영향]. 합쳐서 보고. 속도 3배.

CASE 2 · Parallel – 합의 패턴

같은 질문을 3번 호출 → 다수결. 환각 ↓, 정확도 ↑.

CASE 3 · Orchestrator 구조

```
orchestrator: "여행 계획"
  → worker1: 숙소 추천
  → worker2: 항공권 검색
  → worker3: 일정 짜기
  → orchestrator: 통합 → 최종 보고
```

CASE 4 · Anthropic Deep Research

Anthropic이 출시한 Research 기능이 이 패턴. 마스터 LLM이 주제 분해 → 워커들이 각 영역 조사 → 통합 보고서.

CASE 5 · 비용 통제

Orchestrator는 비용 폭증 위험. 최대 워커 수 + 최대 깊이 + 최대 토큰 3중 제한 필수.

→ Evaluator-Optimizer

품질 우선 작업

번역, 코드, 중요 콘텐츠 — "1번에 끝낼 수 없는" 작업에 강력. 단, 비용 2~5배.

풍부한 사례

CASE 1 · 번역 품질

```
draft = sonnet.translate(text)
score, feedback = sonnet.evaluate(draft, criteria=...)
while score < 8.0 and iter < 3:
    draft = sonnet.translate(text, feedback=feedback)
    score, feedback = sonnet.evaluate(draft, ...)
```

CASE 2 · 코드 자체 리뷰

코드 작성 → 평가자가 "버그·보안·스타일" 체크 → 피드백 → 재작성. PR 품질 ↑.

CASE 3 · 보도자료 작성

초안 → 평가자가 "톤·SEO·간결성" → 피드백 → 보완. 사람 수정 시간 ↓.

CASE 4 · 반복 횟수 제한

무한 루프 차단 위해 최대 3회. 3회 후에도 만족 X면 사람에게 알림.

CASE 5 · 비용 폭탄 1순위

평가 LLM도 토큰. 반복마다 누적. 한 번 계산 후 ROI 확인 필수.

Skills 정의

Anthropic Skills = 함수 + 시스템 프롬프트 + 예시 묶음. **한 번 정의 → 무한 재사용**. Claude Code·Desktop·API에서 동일하게 호출.

풍부한 사례

CASE 1 · 회사 보고서 작성 Skill

Skill: "weekly-report-writer"
- system: "우리 회사 보고서 양식 준수"
- examples: 지난 보고서 5개
- function: format(data) → markdown

CASE 2 · 법무 검토 Skill

"이 계약서를 우리 회사 표준에 맞춰 검토" — 한 번 정의하면 누구나 호출.

CASE 3 · MCP vs Skills

MCP = 외부 시스템 연결 (ERP·DB). Skills = 작업 수행 방법 캡슐화. **둘 다 함께 사용**.

CASE 4 · 사내 표준화

"우리 회사 영문 메일 톤", "고객 응대 스크립트" → Skill로. 직원 누가 써도 동일 톤.

CASE 5 · Skills 마켓플레이스

Anthropic이 공식 Skills 라이브러리 공개. 회사도 사내 Skills 라이브러리 구축 가능.

→ 4 시나리오

Anthropic 권장 4 패턴

백서에서 강조 — 위 5 패턴을 조합한 실전 Agent. 회사 즉시 적용 가능.

풍부한 사례

CASE 1 · Customer Support Agent

흐름: 분류 → 지식베이스 검색 → 답변 → 검증 → 만족도. 못 풀면 사람. **1차 응대 80% 자동화 가능.**

CASE 2 · Coder Agent

흐름: GitHub 이슈 분석 → 코드베이스 탐색 → 영향 파일 찾기 → 수정 → 테스트 → PR. Claude Code 기반.

CASE 3 · Research Agent

흐름: 주제 → 1차 검색·요약 → 누락 영역 식별 → 추가 검색 → 통합 보고서. Anthropic Deep Research가 정확히 이 패턴.

CASE 4 · Operations Agent

흐름: 모니터링 → 이상 감지 → 진단 → 1차 조치 → 사람 알림. 야간 당직 보조.

CASE 5 · 공통 게이트

모두 **사람 confirm 게이트**가 어딘가에 있다. 100% 자율은 위험.

→ 안전·관찰

핵심

Agent는 예측 불가능 → Trace + 비용 + 무한 루프 + 사람 게이트 4중 안전망.

풍부한 사례

CASE 1 · Trace 구조

```
trace = {
  "trace_id": uuid,
  "steps": [
    {"tool": "search", "input": ..., "output": ..., "tokens": 1234},
    {"tool": "draft", ..., "tokens": 5678}
  ],
  "total_tokens": ..., "total_cost": ..., "duration": ...
}
```

CASE 2 · 무한 루프 가드

최대 반복 10회 + 최대 시간 5분 + 최대 비용 \$1. 셋 중 하나라도 초과 시 중단.

CASE 3 · 비용 알림

Trace 단위 누적 비용 모니터링. 평균의 3배 초과 시 알림 + 자동 중단.

CASE 4 · 디버깅 도구

LangSmith, Anthropic Console, Datadog 등으로 trace 시각화.

CASE 5 · 사고 사례

2024년 한 AI 스타트업의 Agent가 무한 루프 → 6시간 만에 \$50K. 가드 없으면 사고 보장.

→ 안티 패턴

Anthropic의 강조

"Workflow로 가능하면 무조건 Workflow." — Anthropic 백서

풍부한 사례

CASE 1 · 단순 작업

"PDF 요약" → 단일 API. Agent 필요 X.

CASE 2 · 고정 순서

"매주 보고서 같은 형식" → cron + 함수. Agent 과잉.

CASE 3 · 실패 비용 큼

금융·법무 발송 → 사람 매 단계 확인. Agent 자율 X.

CASE 4 · 디버그 인프라 없음

Trace·관찰 없으면 사고 시 원인 추적 불가. 도입 안 됨.

CASE 5 · 측정 못함

"잘 됐는지" 객관적으로 측정 못하면 Agent 도입 X. 성공 기준 없는 자동화는 위험.

→ 다음 단계

오늘의 약속

"우리 회사 업무 1개에 5 패턴 중 어떤 게 맞는지" 화이트보드에 그려본다.

로드맵

Lv7: AWS Bedrock·Vertex·Azure 운영 모범 + 한국 인프라 보강.

Lv8: 교육 현장 적용.

Q&A

Q. "Agent 도입 6개월 ROI는?"

A. 명확한 자동화 대상이 있을 때만 6개월 내 양의 ROI. 막연히 도입은 비용만.

Q. "LangChain·LangGraph 써야 하나요?"

A. 작은 프로젝트는 SDK 직접. 복잡해지면 LangGraph 등 프레임워크 고려. Anthropic은 "직접 구현 추천" 백서에서.

Q. "Agent의 결정에 책임은?"

A. Lv3 D4 원칙 그대로 — 최종 책임은 사람. 외부 영향 작업은 사람 게이트 필수.

Q. "Skills와 함수 호출(tool use) 차이?"

A. Tool use는 단일 함수. Skills는 함수+프롬프트+예시 묶음. 더 추상적 단위.